CivicRecords AI

Technical Reference — v1.4.8

Generated May 11, 2026

Apache License 2.0

Open-source, locally-hosted AI that helps American cities respond to open records requests.

Runs entirely on a single machine inside your city's network — no cloud, no vendor lock-in, no resident data leaving the building.

github.com/CivicSuite/civicrecords-ai

Table of Contents

1. Executive Summary

CivicRecords AI is an open-source, locally-hosted AI system purpose-built for municipal open records request processing. Every city in America handles FOIA, CORA, and equivalent state open-records laws. Staff manually search file shares, email archives, and databases — then review every document for exemptions before release. That process is slow, error-prone, and a growing burden as request volumes rise.

No open-source tool existed for the **responder side** of open records at the municipal level. CivicRecords AI fills that gap.

Who It Is For

- City clerks and records officers processing day-to-day open records requests
- Department liaisons providing documents for specific requests
- Legal reviewers and supervisors approving responses before release
- City IT administrators installing and maintaining the system

Key Capabilities

- AI-powered hybrid search (semantic + keyword) over all city documents
- Full request lifecycle management from intake to fulfillment
- Automatic PII and exemption detection with 180 rules across 51 jurisdictions
- Universal connector framework for file systems, REST APIs, and SQL databases
- Hash-chained audit logging designed for legal compliance
- Runs entirely on local hardware — no cloud, no telemetry, no vendor lock-in

Current release: **v1.4.8** (May 2, 2026). 29 database tables, ~30 API endpoints, 627 backend + 36 frontend automated tests. Records-AI consumes CivicCore v0.21.0 for shared schedule validation and next-run computation alongside the earlier CivicCore extractions.

2. System Architecture

CivicRecords AI deploys as a 7-service Docker Compose stack. All services run on Linux containers regardless of host operating system (Windows, macOS, or Linux). No internet connection is required after initial setup.

Service Architecture

The diagram below shows the runtime service topology and primary data flows:

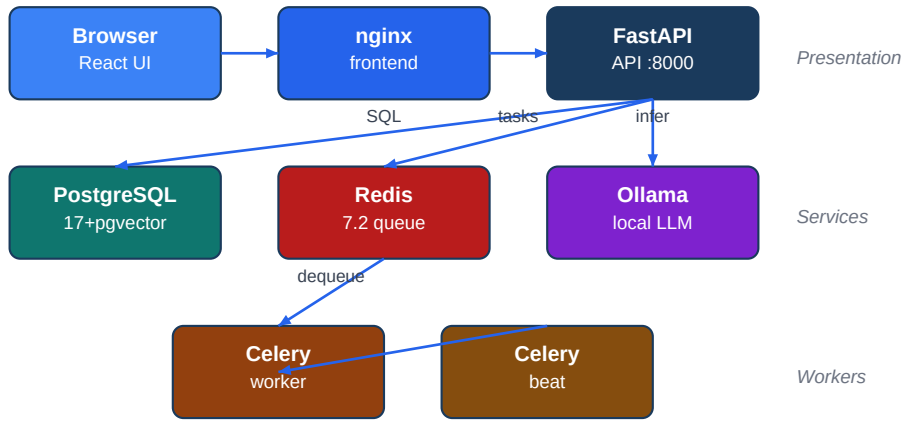
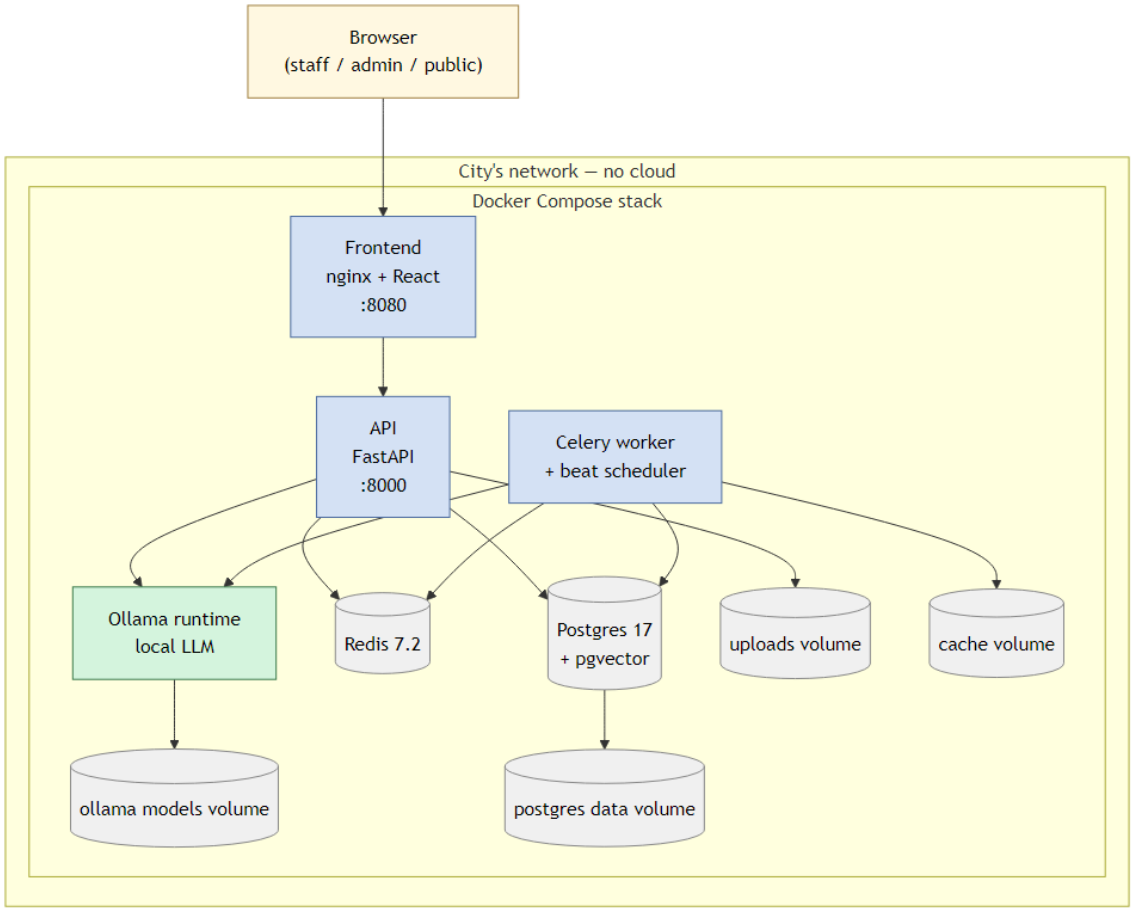


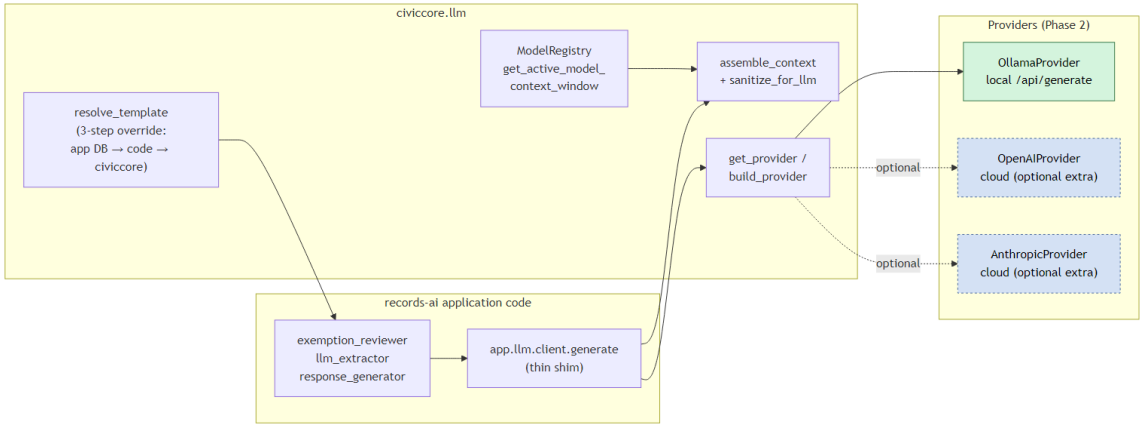
Figure 1 — CivicRecords AI 7-service Docker Compose architecture.

Service	Image	Port	Role
postgres	PostgreSQL 17 + pgvector	5432	Primary data store + vector index
redis	Redis 7.2	6379	Celery task queue and result backend
ollama	Ollama latest	11434	Local LLM inference (Gemma 4)
api	Python 3.12 / FastAPI	8000	REST API, auth, business logic
worker	Celery worker	—	Async document ingestion tasks
beat	Celery beat	—	Scheduled sync trigger (cron)
frontend	nginx + React 18	8080	Staff web interface

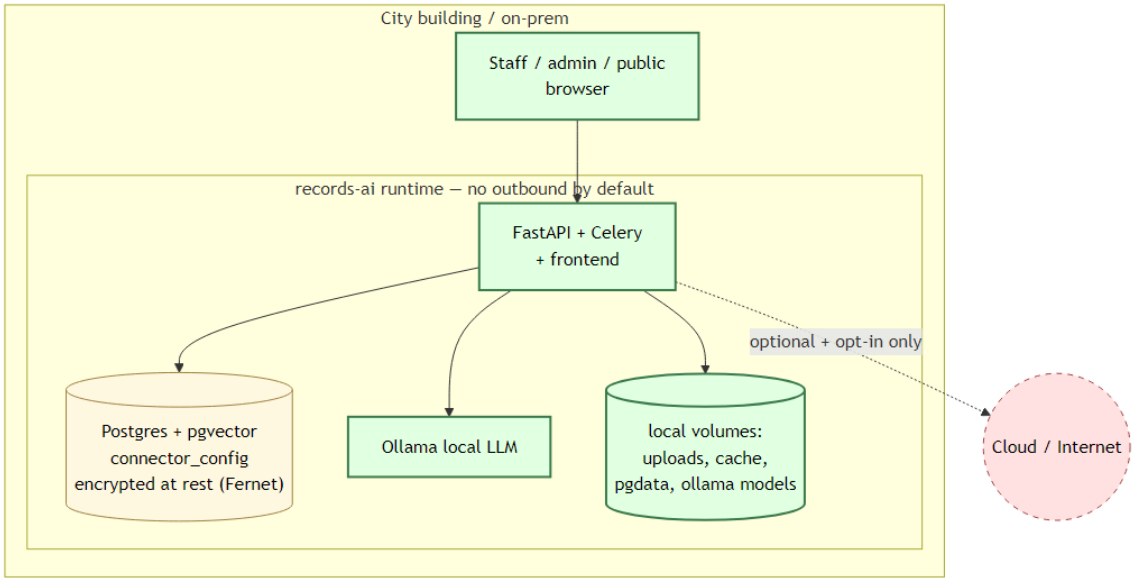
All configuration is via environment variables in `.env`. AMD GPU/NPU hardware auto-detection is included (ROCm on Linux, DirectML on Windows).



Deployment stack — entire system runs inside Docker Compose on the city's network. No cloud, no outbound by default.



LLM call flow: records-ai routes through civiccore.llm to a local Ollama provider; cloud providers are opt-in only.



Sovereignty boundary: all runtime components live inside the city's on-prem network. Connector credentials encrypted at rest with Fernet.

3. Core Features

CivicRecords AI ships with 17 documented features across ingestion, request management, search, exemption detection, analytics, and compliance.

1. AI-Powered Search

Natural language hybrid search (semantic + keyword) across all ingested documents. Source attribution, normalized relevance scores, optional AI-generated summaries.

2. Document Ingestion

Automatic parsing of PDF, DOCX, XLSX, CSV, email, HTML, and text files. Scanned documents processed via multimodal AI (Gemma 4) with Tesseract OCR fallback.

3. Exemption Detection

Tier 1 PII detection (SSN, credit card with Luhn validation, phone, email, bank accounts, state-specific driver's licenses) plus per-state statutory keyword matching. Optional LLM secondary review. All flags require human confirmation.

4. Request Management

Full lifecycle tracking with 11 statuses: intake → clarification → assignment → search → review → drafting → approval → fulfillment → closure. Timeline, messaging, fee tracking, and response letter generation.

5. Guided Onboarding

3-phase wizard helps cities configure their profile, identify data systems across 12 municipal domains, and surface coverage gaps.

6. Municipal Systems Catalog

Curated knowledge base of 25+ municipal software vendors across 12 functional domains (finance, public safety, permitting, HR, etc.) with discovery hints and connector templates.

7. Universal Connector Framework

Standardized protocol (authenticate/discover/fetch/health_check). Ships with file system, REST API (API key/Bearer/OAuth2/Basic; JSON/XML/CSV; page/offset/cursor pagination), and ODBC (SQL databases via pyodbc) connectors.

8. Scheduled Sync & Idempotent Ingestion

Per-source cron scheduling via croniter with 5-minute floor and 7-day min-interval validation. Idempotent pipeline deduplicates by content hash (binary) or stable source-path (structured). Concurrent-update races prevented via SELECT FOR UPDATE.

9. Sync Failure Tracking & Circuit Breaker

Two-layer retry (task-level exponential backoff + record-level per-tick retry). Automatic circuit breaker after 5 consecutive full-run failures. Admin UI with colored health badge, failed records panel, and Sync Now button.

10. Operational Analytics

Real-time metrics: average response time, deadline compliance rate, overdue requests, status breakdown dashboard.

11. Notification Service

Template-based notification system with SMTP email delivery via Celery beat (60s interval). 12 templates across all status transitions.

12. Compliance by Design

Hash-chained audit logs, human-in-the-loop enforcement, AI content labeling, data sovereignty verification. Designed for Colorado CAIA and 50-state regulatory compliance.

13. Civic Design System

Professional UI built with shadcn/ui, civic blue design tokens, sidebar navigation. WCAG 2.2 AA targeted (44px touch targets, skip navigation, icon+color status badges).

14. Federation-Ready

REST API with service accounts enables future cross-jurisdiction record discovery between CivicRecords AI instances.

15. 50-State Exemption Rules

180 exemption rules across 51 jurisdictions (all 50 states + DC) with per-state statutory keyword matching and rule testing with ReDoS protection.

16. Department Access Controls

Staff scoped to own department; admins see all. Department CRUD API with full audit logging and RBAC role hierarchy enforcement.

17. Compliance Templates

5 compliance template documents (AI Use Disclosure, Response Letter Disclosure, CAIA Impact Assessment, AI Governance Policy, Data Residency Attestation) with city profile variable substitution.

4. Technology Stack

Layer	Technology	Version	Notes
Language	Python	3.12	Backend and Celery workers
API	FastAPI	0.115+	Async ASGI framework
ORM	SQLAlchemy	2.0	Async ORM with type annotations
Migrations	Alembic	latest	16 migration scripts
Database	PostgreSQL + pgvector	17	Primary store + vector search
Queue	Redis	7.2	BSD licensed (<8.0.0)
Task runner	Celery	5.x	Worker + beat scheduler
Scheduling	croniter	latest	Apache 2.0, 5-field cron
LLM runtime	Ollama	latest	Local inference, no cloud
LLM model	Gemma 4 / nomic-embed	latest	Recommended configuration
Frontend	React	18	Vite build toolchain
UI library	shadcn/ui + Tailwind CSS	latest	Civic blue design tokens
Auth	JWT + bcrypt	—	6-role RBAC hierarchy
Containers	Docker Compose	v2	7-service stack
Testing	pytest / vitest	latest	627 backend + 36 frontend tests

5. Installation & Quick Start

Requirements

- Docker Desktop (Windows 10/11, macOS 13+) or Docker Engine (Linux)
- 8+ CPU cores, 32 GB RAM, 50 GB free disk space
- No internet connection required after initial setup

Install (Windows)

```
git clone https://github.com/CivicSuite/civicrecords-ai.git
cd civicrecords-ai
.\install.ps1
```

Install (macOS / Linux)

```
git clone https://github.com/CivicSuite/civicrecords-ai.git
cd civicrecords-ai
bash install.sh
```

First Use

1. Open http://localhost:8080 in your browser
2. Sign in with the admin credentials configured in .env
3. Go to Sources → Add Source → enter a directory path to your documents
4. Click Ingest Now — documents are parsed, chunked, and indexed automatically
5. Go to Search — type a natural language query and get cited results

Key Environment Variables

Variable	Description	Default
DATABASE_URL	PostgreSQL connection string	postgres://civicrecords:...
JWT_SECRET_FILE	Mounted secret file for JWT tokens	/run/secrets/jwt_secret
FIRST_ADMIN_EMAIL	Initial admin account email	admin@example.gov
FIRST_ADMIN_PASSWORD_FILE	Mounted first-admin password file	/run/secrets/first_admin_password
CIVICRECORDS_SECRET_DIR	Host directory for Docker secret files	./data/secrets
OLLAMA_BASE_URL	Ollama API endpoint	http://ollama:11434
REDIS_URL	Redis connection string	redis://redis:6379/0
AUDIT_RETENTION_DAYS	Audit log retention period	1095 (3 years)
SMTP_HOST	SMTP server for email notifications	(optional)

6. Database Schema

CivicRecords AI uses 29 PostgreSQL tables managed by 16 Alembic migration scripts. pgvector extension provides the embeddings column used for semantic search.

Table	Description
users	Staff accounts with RBAC roles and department associations
departments	City department definitions for access scoping
audit_logs	Hash-chained, tamper-evident activity log
requests	Open records request lifecycle tracking
request_documents	Documents attached to requests with include/exclude decisions
request_messages	Internal and requester communications per request
request_fees	Fee line items, waivers, and collection status
request_events	Timeline events for request audit trail
documents	Ingested document metadata and content hashes
document_chunks	Text segments from parsed documents
embeddings	pgvector floating-point vectors for semantic search
datasources	Connector configurations and sync state
sync_failures	Per-record failure tracking with retry and dismiss workflow
sync_run_log	One-row-per-run ingestion history
exemption_rules	State-specific statutory exemption rules
exemption_flags	Per-document flags awaiting human review
exemption_outcomes	Accepted/rejected flag decisions for auditability
onboarding_state	Wizard progress and city profile data
city_profile	City name, state, contact, statutory deadline
municipal_systems	Known vendor catalog entries
coverage_items	Gap map entries per functional domain
service_accounts	API keys for federation access
notification_templates	Email template definitions
notification_log	Delivery history for all notifications
compliance_templates	5 AI/compliance disclosure document templates
model_registry	LLM model metadata and compliance records
llm_interactions	Logged LLM calls with token counts

settings	System-wide configuration key-value store
alembic_version	Database migration version tracking

7. API Reference Summary

The FastAPI backend exposes approximately 30 REST endpoints under the `/api/v1/` prefix. Full OpenAPI docs available at `http://localhost:8000/docs` when running.

Prefix	Description
/api/v1/auth	Login, token refresh, logout
/api/v1/users	User CRUD, role management (admin only)
/api/v1/departments	Department CRUD, access scoping
/api/v1/requests	Full request lifecycle (create, status transitions, fees, messages)
/api/v1/documents	Document metadata, chunk retrieval
/api/v1/search	Hybrid semantic + keyword search with filters
/api/v1/datasources	Connector CRUD, sync triggers, health status
/api/v1/ingestion	Ingestion job status, error logs
/api/v1/exemptions	Rules CRUD, flag review, auditability dashboard
/api/v1/onboarding	3-phase wizard endpoints, city profile
/api/v1/compliance	Template render, model registry
/api/v1/audit	Audit log export (CSV/JSON)
/api/v1/analytics	Dashboard metrics, deadline compliance
/health	Service health check

8. Connector Framework

All data source connectors implement a standardized 4-method protocol. This ensures consistent behavior, testability, and future extensibility.

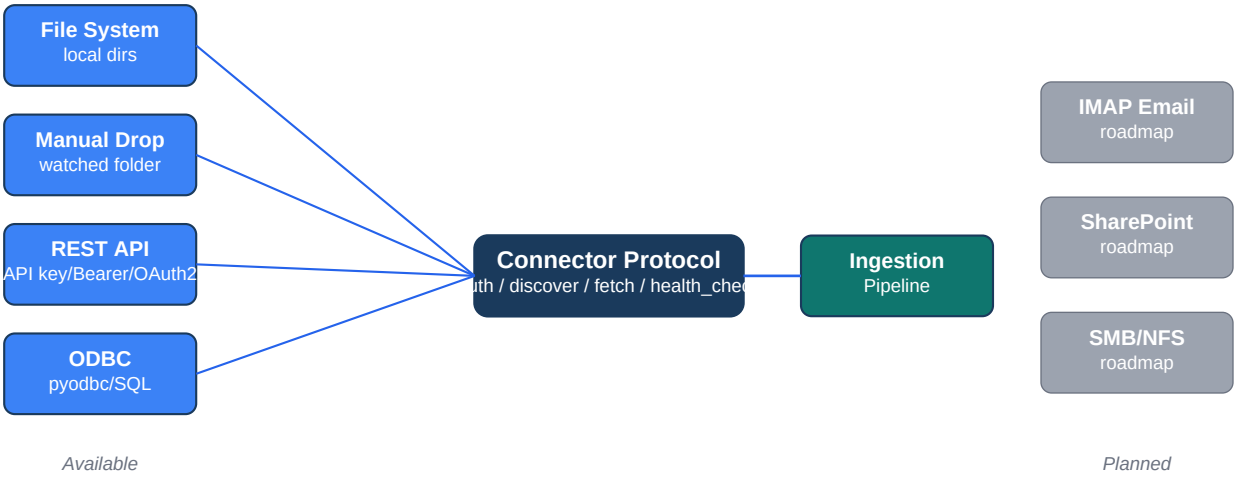


Figure 2 — Connector Framework: protocol layer with available and planned connectors.

Method	Description
authenticate()	Validate credentials. Returns success/error with human-readable message.
discover()	List available records. Returns list of {id, source_path} dicts.
fetch(source_path)	Retrieve a single record's content. Returns text or binary.
health_check()	Lightweight liveness check. Used by the data source card health badge.

Available Connectors

- File System — local and network directories. Most common source type.
- Manual Drop — watched drop folder for files exported manually by staff. Use when a system has no API. Config key: drop_path.
- REST API — API key, Bearer token, OAuth2 client-credentials, Basic auth. JSON/XML/CSV response formats. Page/offset/cursor pagination. data_key dotted-path extraction.
- ODBC — SQL databases via pyodbc. Row-as-document with SQL-injection guards. Primary key encoding/decoding with special-character support.

Planned Connectors

- IMAP Email — Exchange and Gmail/Google Workspace integration
- SharePoint — Graph API with Azure AD authentication
- SMB/NFS — Direct network share access

9. Sync Failure & Circuit Breaker

CivicRecords AI implements a two-layer retry model designed to distinguish transient infrastructure failures from persistent data problems, and to protect the system from cascading failures on unreliable municipal APIs.

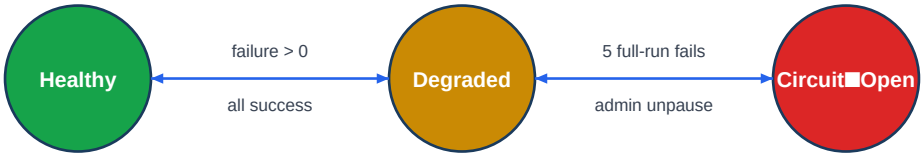


Figure 3 — Health state machine: Healthy → Degraded → Circuit Open.

Retry Layers

Layer	Trigger	Behavior
Task-level	Transient error (IOError, Ollama timeout)	3 retries: 30s → 90s → 270s. Max 10 min cap.
Record-level	Task exhaustion → sync_failure row	Per-tick retry. Cap: N=100 OR T=90s per run.

Circuit Breaker Rules

- Circuit opens after 5 consecutive full-run failures (authenticate or discover throws, OR all fetches fail)
- Zero-work runs (0 records discovered, no retries) do NOT increment the counter
- Any single success resets the counter to 0
- Admin unpause triggers a 2-failure grace period threshold before returning to threshold=5
- health_status computed live at response time: healthy / degraded / circuit_open

10. Security Architecture

Authentication

JWT tokens with bcrypt password hashing. Login rate limiting. Admin-only user creation.

Authorization

6-role RBAC hierarchy enforced at API layer. Department-level access controls for staff.

Credentials

Connector credentials encrypted AES-256 at rest. Never logged, exported, or returned.

Audit Logging

Hash-chained audit logs — every action, every user, every timestamp. CSV/JSON export. 3-year retention default.

Human-in-the-Loop

No auto-redaction, no auto-denial, no auto-release. All AI suggestions require human confirmation.

Data Sovereignty

Runs entirely on local hardware. No telemetry, analytics, or crash reporting. All LLM inference via Ollama.

Prompt Injection

Central LLM client with prompt injection sanitization. ReDoS protection on all admin-entered regex.

CJIS Compliance

Compliance gate enforced for public safety connectors.

Macro Stripping

VBA macros stripped from all ingested DOCX/XLSX files.

AI Labeling

All AI-generated content is explicitly labeled. Labels cannot be removed by staff.

11. Key Engineering Decisions

The following decisions constrain implementation and are each proven by an automated test. Per the canonical spec: if you want to change a decision, update the test first.

ID	Decision	Rationale
D-IDEM-1	Split idempotency: binary by (source_id, file_hash); structured by (source_id, source_path).	REST/ODBC hash is not deterministic.
D-IDEM-7	On UPDATE: DELETE existing Chunk rows and pgvector embeddings in a transaction before re-generating.	Stale embeddings are a correctness bug.
D-IDEM-8	ingest_structured_record uses SELECT … FOR UPDATE before comparing hashes to prevent concurrent-worker race	
D-SCHED-1	sync_schedule (cron/croniter) replaces schedule_minutes.	Correct trigger: get_next(datetime) <= now(). Original spec had inver
D-SCHED-2	Min interval validated via rolling 7-day sample (2016 intervals).	Floor: 5 minutes. Catches adversarial cron expressions.
D-SCHED-3	Cron evaluated in UTC.	UI shows both UTC and local time. Wizard discloses 'All schedules r
D-FAIL-1	Two retry layers: task-level (3 retries, 30s→90s→270s) for transient errors; record-level (N=100/T=90s cap) for persiste	
D-FAIL-2	Partial failure: cursor advances past successful records.	Failed records written to sync_failures only.
D-FAIL-4	Circuit breaker: full-run failure = authenticate() or discover() throws OR all fetches fail.	Zero-work does NOT increment counter.
D-FAIL-5	Unpause grace: threshold=2 for first post-unpause window, reverts to 5 after success.	Prevents 5-cycle wait confusion.
D-FAIL-6	Dismiss = soft delete (status=dismissed + dismissed_at + dismissed_by).	Hard delete prohibited — compliance artifact.
D-FAIL-8	health_status computed at response time via LEFT JOIN — not stored.	Avoids cache staleness.
D-FAIL-12	429 with Retry-After honored at task-level (not sync_failures).	Capped at 600s to prevent worker starvation.
D-FAIL-13	sync_failures and sync_run_log both CASCADE on DataSource delete..	
D-UI-1	Sync Now button stays disabled until last_sync_at advances (exponential backoff polling: 5s→10s→20s→30s).	Timeout 15 min.
D-UI-2	Circuit-open notifications: created_by recipient, fallback to ADMIN role users.	Rate-limit: 5-min digest.
D-TENANT-1	Single-tenant per install (one city per deployment).	No org-level isolation within a deployment.

12. Test Suite

CivicRecords AI ships 627 backend tests across 45+ pytest modules and 36 frontend component tests. Tests are a first-class artifact — every key engineering decision is enforced by a named test function.

Module Group	Count	Coverage Focus
Auth & users	38	JWT, RBAC, rate limiting, admin-only creation
Requests lifecycle	52	Status transitions, fees, messages, timeline
Exemption detection	47	PII regex, Luhn, state rules, ReDoS protection
Ingestion pipeline	61	Idempotency, dedup, concurrent update, chunk atomicity
Connector framework	44	REST, ODBC, file system, auth methods, pagination
Scheduler	28	Cron validation, UTC disclosure, adversarial exprs
Sync failure / CB	55	Retry layers, circuit breaker, dismiss workflow
Audit logging	19	Hash chain, retention, export
Analytics & notifs	31	Dashboard metrics, SMTP delivery, templates
Onboarding / catalog	22	3-phase wizard, gap map, systems catalog
Compliance & models	16	Template render, model registry, CAIA
Migrations	10	Schema correctness, backfill logic
Frontend (vitest)	5	DataSourceCard sync button, component rendering

Running Tests

```
# Unit tests (no Docker required)
cd backend && python -m pytest tests/ -v

# Integration tests (requires Docker)
docker compose up -d postgres redis
DATABASE_URL=postgresql+asyncpg://civicrecords:civicrecords@localhost:5432/civicrecords_test \
python -m pytest tests/ -v

# Frontend tests
cd frontend && npm run test
```

13. License

CivicRecords AI is released under the **Apache License 2.0**.

All dependencies use permissive (MIT, Apache 2.0, BSD) or weak-copyleft (LGPL, MPL) licenses. No AGPL, SSPL, or BSL dependencies. Redis is pinned to <8.0.0 (BSD licensed; 8.x changed licensing).

For complete documentation, source code, and issue tracking, see:
github.com/CivicSuite/civicrecords-ai